

Here's what the stack looks like at its deepest point, when n equals 1. The bottom of the stack is the oldest call, the top is the newest.

TOP factorial $n=1$ returns to: $n * \text{factorial}(n-1)$ inside factorial(2)

dynamic link: factorial(2)'s frame

static link: factorial(2)'s frame

factorial $n=2$ returns to: $n * \text{factorial}(n-1)$ inside factorial(3)

dynamic link: factorial(3)'s frame

static link: factorial(3)'s frame

factorial $n=3$ returns to: $n * \text{factorial}(n-1)$ inside factorial(4)

dynamic link: factorial(4)'s frame

static link: factorial(4)'s frame

BOTTOM factorial $n=4$ returns to: the original call factorial(4) at the top level

dynamic link: None, nothing called it

static link: None

Since factorial isn't nested inside anything else, every static link and dynamic link end up pointing at the exact same frame, whoever called it.

Now here's the unwind, one step at a time:

Step 1: $n=1$ finishes and returns 1 to $n=2$.

Stack left: $n=4$, $n=3$, $n=2$

$n=2$ does 2 times 1, gets 2, and returns.

Step 2: $n=2$ returns 2 to $n=3$.

Logan Titzer

CS379

PA06 – Activation Record

Stack left: $n=4$, $n=3$

$n=3$ does 3 times 2, gets 6, and returns.

Step 3: $n=3$ returns 6 to $n=4$.

Stack left: $n=4$

$n=4$ does 4 times 6, gets 24, and returns.

Step 4: $n=4$ returns 24 back to the original call.

Stack is now empty.

2. Static Link vs. Dynamic Link

Here's a USILang example where one function is defined inside another.

```
func outer() {  
    func inner() { return x; }  
    call relay();  
}  
  
func relay() {  
    call inner();  
}  
  
call outer();
```

inner is written inside outer, so its static link is always outer's frame since that's the scope it should be looking things up in. But outer never calls inner directly in this example, it calls relay, and relay is the one that actually calls inner. So inner's dynamic link ends up pointing to relay's frame instead of outer's. That's the whole point here, static and dynamic point at two different frames for the same call.

Static links matter because dynamic links only tell you who called you right now, not where the function was actually written. If inner needs some variable that isn't local to it, it has to look it up in the scope it was defined in, which is outer, no matter who called it this time

Logan Titzer

CS379

PA06 – Activation Record

around. Without a static link there'd be no way to find that back once you're more than one call removed, which is exactly what happens in this relay example.

3. Stack Overflow, Precisely

Say the stack has S bytes total and each activation record takes up R bytes. The max safe recursion depth is just how many of those records can fit before you run out:

$$\text{max depth} < S / R$$

Once you'd need one more frame than that fits, you overflow.

Tail recursion doesn't get around this in a naive tree-walking interpreter because the interpreter pushes a brand new frame for every recursive call no matter where in the function that call happens. It's just walking the tree and treating every call the same way, it doesn't notice or care that the recursive call happens to be the last thing the function does. A real compiler can catch that and reuse the same frame instead of stacking a new one every time, but that's a specific optimization someone has to build in. Without it, a tail-recursive call costs just as much stack space as any other recursive call.